

Правила ESLint

Данные правила позволяют сделать наш код чистым. Редактор автоматически уведомляет при возникновении той или иной ошибки.

Необходим пробел до и после ключевого слова

Тип: Предупреждение

Название: keyword-spacing

[Источник](#)

Ошибка:

Перед ключевым словом или после него не проставлен пробел.

Решение:

Необходимо перед ключевым словом поставить пробел.

Пример:

- Код с ошибкой

```
1 function ДнейМесяца(Дата) {
2     if (Дата) {
3         Дата = new Date(Дата);
4     }else{ // нужны пробелы до и после else
5         Дата = new Date();
6     }
7     return new Date(new Date(Дата.getFullYear(), Дата.getMonth() + 1,
8     Дата.getDate()).setDate(0)).getDate()
9 }
```

- Код без ошибки

```
1 function ДнейМесяца(Дата) {
2     if (Дата) {
3         Дата = new Date(Дата);
4     } else {
5         Дата = new Date();
6     }
7     return new Date(new Date(Дата.getFullYear(), Дата.getMonth() + 1,
8     Дата.getDate()).setDate(0)).getDate()
9 }
```

Запрещен пробел перед скобкой функции

Тип: Предупреждение

Название: space-before-function-paren

[Источник](#)

Ошибка:

Поставлен пробел перед скобкой функции.

Решение:

Необходимо перед скобкой функции удалить пробел.

Пример:

- Код с ошибкой

```
1 function ДнейМесяца (Дата) { // пробел перед скобкой функции
2     if (Дата) {
```

```

3      Дата = new Date(Дата);
4  } else {
5      Дата = new Date();
6  }
7  return new Date(new Date(Дата.getFullYear(), Дата.getMonth() + 1,
Дата.getDate()).setDate(0)).getDate()
8  }

```

- Код без ошибки

```

1  function ДнейМесяца(Дата) {
2      if (Дата) {
3          Дата = new Date(Дата);
4      } else {
5          Дата = new Date();
6      }
7      return new Date(new Date(Дата.getFullYear(), Дата.getMonth() + 1,
Дата.getDate()).setDate(0)).getDate()
8  }

```

Запрещены пробелы в конце строк

Тип: Предупреждение

Название: no-trailing-spaces

[Источник](#)

Ошибка:

В конце строки добавлены лишние пробелы.

Решение:

Удалить лишние пробелы в конце строки.

Пример:

- Код с ошибкой

```

1  var foo = 0;//.....
2  var baz = 5;//..

```

- Код без ошибки

```

1  var foo = 0;
2  var baz = 5;

```

Запрещено использовать пробелы внутри круглых скобок

Тип: Предупреждение

Название: space-in-parens

[Источник](#)

Ошибка:

После открытия и перед закрытием круглой скобки присутствуют пробелы.

Решение:

Необходимо удалить лишние пробелы.

Пример:

- Код с ошибкой

```

1  foo( 1 + 2 );
2  foo(1 + 2 );

```

```
3 | foo( 1 + 2 );
```

- Код без ошибки

```
1 | foo(1 + 2);
```

Требуется пробел вокруг логических операторов

Тип: Предупреждение

Название: space-infix-ops

[Источник](#)

Ошибка:

Пропущен пробел до и после логического оператора.

Решение:

Добавить пробелы до и после логического оператора.

Пример:

- Код с ошибкой

```
1 | a+b
2 | a?b:c
```

- Код без ошибки

```
1 | a + b
2 | a ? b : c
```

Требуется пробел перед блоками {}

Тип: Предупреждение

Название: space-before-blocks

[Источник](#)

Ошибка:

Пропущен пробел перед блоками {}.

Решение:

Добавить пробел перед блоками {}.

Пример:

- Код с ошибкой

```
1 | if (a){
2 |     b();
3 | }
```

- Код без ошибки

```
1 | if (a) {
2 |     b();
3 | }
```

Запрещено использовать табуляцию и пробелы одновременно

Тип: Предупреждение

Название: no-mixed-spaces-and-tabs

[Источник](#)

Ошибка:

В отмеченной строке одновременно используются табуляция и пробел.

Решение:

Всегда использовать только пробелы.

Пример:

- Код с ошибкой

```
1  if (Наим.СчетУчета && ЭтоЗабалансовыйСчет(Наим.СчетУчета)) {
2  /*...*/if (Комиссия) {
3  /*...-->*/if (!Наим.Собственник) { // нельзя использовать в одной
   строке табуляцию и пробелы
4  /*.....*/Ошибка('Не указан собственник у ' + Наим.Наименование);
5  /*...*/}
6  /*...*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад,
   Наим.Собственник], Нет, [], пСтоимость);
7  /*...*/} else {
8  /*....*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад], Нет, [],
   пСтоимость);
9  /*...*/}
10 }
```

- Код без ошибки

```
1  if (Наим.СчетУчета && ЭтоЗабалансовыйСчет(Наим.СчетУчета)) {
2  /*...*/if (Комиссия) {
3  /*...*/if (!Наим.Собственник) {
4  /*.....*/Ошибка('Не указан собственник у ' + Наим.Наименование);
5  /*....*/}
6  /*....*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад,
   Наим.Собственник], Нет, [], пСтоимость);
7  /*...*/} else {
8  /*....*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад], Нет, [],
   пСтоимость);
9  /*...*/}
10 }
```

Отступ вложенных блоков и операторов должен быть кратный трем пробелам

Тип: Предупреждение

Название: indent

[Источник](#)

Ошибка:

Отступ не кратен трем пробелам.

Решение:

Всегда использовать отступ, кратный трем пробелам.

Пример:

- Код с ошибкой

```
1  if (Наим.СчетУчета && ЭтоЗабалансовыйСчет(Наим.СчетУчета)) {
2  /*...*/if (Комиссия) {
3  /*...*/if (!Наим.Собственник) { // отступ не кратен трем
4  /*.....*/Ошибка('Не указан собственник у ' + Наим.Наименование);
5  /*...*/}
6  /*...*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад,
   Наим.Собственник], Нет, [], пСтоимость);
7  /*...*/} else {
```

```

8  /*...*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад], Нет, [],
   пСтоимость);
9  /*...*/}
10 }

```

- Код без ошибки

```

1  if (Наим.СчетУчета && ЭтоЗабалансовыйСчет(Наим.СчетУчета)) {
2  /*...*/if (Комиссия) {
3  /*...*/if (!Наим.Собственник) {
4  /*.....*/Ошибка('Не указан собственник у ' + Наим.Наименование);
5  /*...*/}
6  /*...*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад,
   Наим.Собственник], Нет, [], пСтоимость);
7  /*...*/} else {
8  /*...*/Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад], Нет, [],
   пСтоимость);
9  /*...*/}
10 }

```

Неправильные пробелы

Тип: Ошибка

Название: no-irregular-whitespace

[Источник](#)

Это правило запрещает использование следующих символов, кроме тех, где это разрешено параметрами:

- \u000B — Line Tabulation (\v) — <VT>
- \u000C — Form Feed (\f) — <FF>
- \u00A0 — No-Break Space — <NBSP>
- \u0085 — Next Line
- \u1680 — Ogham Space Mark
- \u180E — Mongolian Vowel Separator — <MVS>
- \ufeff — Zero Width No-Break Space — <BOM>
- \u2000 — En Quad
- \u2001 — Em Quad
- \u2002 — En Space — <ENSP>
- \u2003 — Em Space — <EMSP>
- \u2004 — Tree-Per-Em
- \u2005 — Four-Per-Em
- \u2006 — Six-Per-Em
- \u2007 — Figure Space
- \u2008 — Punctuation Space — <PUNCSP>
- \u2009 — Thin Space
- \u200A — Hair Space
- \u200B — Zero Width Space — <ZWSP>
- \u2028 — Line Separator
- \u2029 — Paragraph Separator
- \u202F — Narrow No-Break Space
- \u205f — Medium Mathematical Space
- \u3000 — Ideographic Space

Ошибка:

Скрытые символы могут быть случайно перенесены в редактор при копировании кода из внешнего источника.

Решение:

Удалить все скрытые символы.

Пример:

- Код с ошибкой

```
1 function thing/*<MVS>*//(){ // символ <MVS> невидим в редакторе
2     return 'test';
3 }
```

- Код без ошибки

```
1 function thing (){
2     return 'test';
3 }
```

Присвоение вместо условного выражения

Тип: Ошибка

Название: no-cond-assign

[Источник](#)

Ошибка:

Допущена ошибка при использовании условного оператора — используется присвоение.

Решение:

Проверьте условие в операторе.

Пример:

- Код с ошибкой

```
1 if (пСистемаН0 = 'УСН') { // необходимо сравнить, а не присвоить
2     var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);
3 }
```

- Код без ошибки

```
1 if (пСистемаН0 == 'УСН') {
2     var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);
3 }
```

Требовать === и !==, вместо == и !=

Тип: Предупреждение

Название: eqeqeq

[Источник](#)

Ошибка:

Использование не безопасных операторов равенства == и !=

Решение:

Использовать безопасные операторы равенства === и !==

Пример:

- Код с ошибкой

```
1 if (x == 42) { }
2 if (" " == text) { }
3 if (obj.getStuff() != undefined) { }
```

- Код без ошибки

```
1 if (x === 42) { }
2 if (" " === text) { }
3 if (obj.getStuff() !== undefined) { }
```

Константы в условных выражениях запрещены

Тип: Ошибка

Название: no-constant-condition

[Источник](#)

Ошибка:

В условном выражении не указан оператор сравнения.

Решение:

Не используйте выражения, значения которых не зависят от остальной функции.

Пример:

- Код с ошибкой

```
1  if ('УСН') { // необходима проверка вместо константы
2      var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);
3  }
```

- Код без ошибки

```
1  if (пСистемаНО == 'УСН') {
2      var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);
3  }
```

Запрещен вызов консоли

Тип: Ошибка

Название: no-console

[Источник](#)

Ошибка:

Прикладной код СБИС не поддерживает работу с консолью браузера.

Решение:

Не используйте вызов консоли.

Пример:

- Код с ошибкой

```
1  function НоваяФункция(){
2      console.log("Записать сообщение об уровне ошибки");
3  }
```

- Код без ошибки

```
1  Не вызывать консоль
```

Запрещено использовать функцию eval

Тип: Ошибка

Название: no-eval

[Источник](#)

Ошибка:

eval() – JavaScript-функция, потенциально опасна и часто используется неправильно. Использование eval() может открыть программу для различных атак путем инъекций.

Решение:

Не использовать функцию eval().

Пример:

- Код с ошибкой

```
1  if (пСистемаНО == 'УСН') {
```

```
2   var оУСНДоходы = eval(Код + ';УСНДоходы(пДатаН, пДатаК);'); //  
   нельзя использовать eval  
3 }
```

- Код без ошибки

```
1  if (пСистемаНО == 'УСН') {  
2      var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);  
3  }
```

Вызовы отладчика запрещены

Тип: Ошибка

Название: no-debugger

[Источник](#)

Ошибка:

СБИС не поддерживает метод отладки.

Решение:

Не используйте отладчик в редакторе.

Пример:

- Код с ошибкой

```
1  if (пСистемаНО == 'УСН') {  
2      debugger; // вызовы отладчика запрещены  
3      var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);  
4  }
```

- Код без ошибки

```
1  if (пСистемаНО == 'УСН') {  
2      var оУСНДоходы = УСНДоходы(пДатаН, пДатаК);  
3  }
```

Запрещено удалять переменные

Тип: Ошибка

Название: no-delete-var

[Источник](#)

Ошибка:

Использование оператора delete.

Решение:

Убрать оператор удаления из кода.

Пример:

- Код с ошибкой

```
1  function ПрошлыйБухОтчет(Годовой) {  
2      var ПредОтчет = ОтчетПрошлогоПериода();  
3  
4      if (!Годовой) {  
5          if (ПредОтчет.hasOwnProperty('ОтчетИзмКап')) {  
6              delete ПредОтчет; // нельзя удалять переменные  
7          }  
8      }  
9      return ПредОтчет;  
10 }
```

- Код без ошибки

```
1  function ПрошлыйБухОтчет(Годовой) {
```



```

2      var ПредОтчет = ОтчетПрошлогоПериода();
3
4      if (!Годовой) {
5          if (ПредОтчет.hasOwnProperty('ОтчетИзмКап')) {
6              delete ПредОтчет.ОтчетИзмКап;
7          }
8      }
9      return ПредОтчет;
10 }

```

Дублирующийся аргумент

Тип: Ошибка

Название: no-dupe-args

[Источник](#)

Ошибка:

В определениях функций повторяются аргументы.

Решение:

Проверьте определение функции. Удалите или замените повторяющиеся аргументы.

Пример:

- Код с ошибкой

```

1  function ЗаполнитьРаздел3РасчетУбытков(ВсеУбыт, ДоходыРасходы,
2  ВсеУбыт) { // нельзя дублировать имена аргументов
3      if (ОтчетКвартальный)
4          return {};
5      else
6          return {
7          'СумУбытПредПер': УбыткиПоГодам(ВсеУбыт.pre_year_losses),
8          'НалБаза': ДоходыРасходы.Дох,
9          'СумУбытФактУм': ВсеУбыт.adjust_loss,
10         'СумУбытТек': ДоходыРасходы.Убыт,
11         'СумУбытТекПер': УбыткиПоГодам(ВсеУбыт.next_year_losses,
12         ДоходыРасходы.Убыт, ВсеУбыт)
13     };
14 }

```

- Код без ошибки

```

1  function ЗаполнитьРаздел3РасчетУбытков(ВсеУбыт, ДоходыРасходы,
2  Превышение) {
3      if (ОтчетКвартальный)
4          return {};
5      else
6          return {
7          'СумУбытПредПер': УбыткиПоГодам(ВсеУбыт.pre_year_losses),
8          'НалБаза': ДоходыРасходы.Дох,
9          'СумУбытФактУм': ВсеУбыт.adjust_loss,
10         'СумУбытТек': ДоходыРасходы.Убыт,
11         'СумУбытТекПер': УбыткиПоГодам(ВсеУбыт.next_year_losses,
12         ДоходыРасходы.Убыт, Превышение)
13     };
14 }

```

Дублирование ключа

Тип: Предупреждение

Название: no-dupe-keys

[Источник](#)

Ошибка:

Ключ в объекте продублирован.

Решение:

Проверьте имена ключей в коде. Измените или удалите дублирующийся ключ.

Пример:

- Код с ошибкой

```
1 var НаименованиеА0 = {
2   'СуммаСебест': ДокументРасширение.Сумма,
3   'СуммаСебестБезНДС': ДокументРасширение.Сумма,
4   'Тип': 10,
5   'СчетРасхода': СчетЗатрат,
6   'Аналитика1': Аналитика1,
7   'Аналитика1': Аналитика2 // нельзя дублировать ключи
8 };
```

- Код без ошибки

```
1 var НаименованиеА0 = {
2   'СуммаСебест': ДокументРасширение.Сумма,
3   'СуммаСебестБезНДС': ДокументРасширение.Сумма,
4   'Тип': 10,
5   'СчетРасхода': СчетЗатрат,
6   'Аналитика1': Аналитика1,
7   'Аналитика2': Аналитика2
8 };
```

Повторяющаяся метка

Тип: Предупреждение

Название: no-duplicate-case

[Источник](#)

Ошибка:

Происходит дублирование метки.

Решение:

Проверьте использование меток. Удалите дубли.

Пример:

- Код с ошибкой

```
1 function ОпределитьСтатьюРасхода(Счет) {
2   if (ПлатежныеДокументы.Операция)
3     return ПлатежныеДокументы.Операция;
4   switch (Счет) {
5     case Счета.НалогНаИмущество: return
      НайтиСтатьюРасхода('Имуц');
6     case Счета.ЗемельныйНалог: return
      НайтиСтатьюРасхода('ЗемНал');
7     case Счета.ЗемельныйНалог: return
      НайтиСтатьюРасхода('ТранспНал'); // нельзя дублировать метки
8     case Счета.ТорговыйСбор: return
      НайтиСтатьюРасхода('ТоргСбор');
9     case Счета.НДФЛ: return НайтиСтатьюРасхода('ТрудНал');
10  }
11  return (Счет.indexOf('69-') < 0) ? НайтиСтатьюРасхода('ПрочНал')
```

```
12 : НайтиСтатьюРасхода('ТрудНал');  
12 }
```

- Код без ошибки

```
1 function ОпределитьСтатьюРасхода(Счет) {  
2     if (ПлатежныеДокументы.Операция)  
3         return ПлатежныеДокументы.Операция;  
4     switch (Счет) {  
5         case Счета.НалогНаИмущество: return  
        НайтиСтатьюРасхода('Имущ');  
6         case Счета.ЗемельныйНалог: return  
        НайтиСтатьюРасхода('ЗемНал');  
7         case Счета.ТранспортныйНалог: return  
        НайтиСтатьюРасхода('ТранспНал');  
8         case Счета.ТорговыйСбор: return  
        НайтиСтатьюРасхода('ТоргСбор');  
9         case Счета.НДФЛ: return НайтиСтатьюРасхода('ТрудНал');  
10    }  
11    return (Счет.indexOf('69-') < 0) ? НайтиСтатьюРасхода('ПрочНал')  
12    : НайтиСтатьюРасхода('ТрудНал');  
12 }
```

Пропущена точка с запятой в конце строки

Тип: Предупреждение

Название: semi

[Источник](#)

Ошибка:

В коде пропущена точка с запятой.

Решение:

Добавьте пропущенную точку с запятой в конце строки.

Пример:

- Код с ошибкой

```
1 var ПДНалог = Документ.ПДНалог // пропущена точка с запятой после  
  объявления переменной  
2 ПолучитьНомерСчетаПоИД(ПДНалог.СчетУчета) // пропущена точка с  
  запятой после вызова функции  
3 ВидПлатежаПоТипу(ПДНалог.ТипПлатежа) // пропущена точка с запятой  
  после вызова функции
```

- Код без ошибки

```
1 var ПДНалог = Документ.ПДНалог;  
2 ПолучитьНомерСчетаПоИД(ПДНалог.СчетУчета);  
3 ВидПлатежаПоТипу(ПДНалог.ТипПлатежа);
```

Вложенный блок избыточен

Тип: Предупреждение

Название: no-lone-blocks

[Источник](#)

Ошибка:

В коде содержатся лишние фигурные скобки.

Решение:

Удалить лишние фигурные скобки.

Пример:

- Код с ошибкой

```
1  if (Наим.СчетУчета && ЭтоЗабалансовыйСчет(Наим.СчетУчета)) {
2      { // лишняя фигурная скобка
3          if (Комиссия) {
4              if (!Наим.Собственник) {
5                  Ошибка('Не указан собственник у ' +
Наим.Наименование);
6              }
7              Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад,
Наим.Собственник], Нет, [], пСтоимость);
8          } else
9              Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад], Нет,
[], пСтоимость);
10     } // лишняя фигурная скобка
11 }
```

- Код без ошибки

```
1  if (Наим.СчетУчета && ЭтоЗабалансовыйСчет(Наим.СчетУчета)) {
2      if (Комиссия) {
3          if (!Наим.Собственник) {
4              Ошибка('Не указан собственник у ' + Наим.Наименование);
5          }
6          Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад,
Наим.Собственник], Нет, [], пСтоимость);
7      } else
8          Проводка(ДатаПроводки, Наим.СчетУчета, [Наим.Склад], Нет, [],
пСтоимость);
9  }
```

Переменная уже объявлена

Тип: Предупреждение

Название: no-redeclare

[Источник](#)

Ошибка:

Определена переменная с повторяющимся именем.

Решение:

Удалите команду объявления переменной.

Пример:

- Код с ошибкой

```
1  function ПолучитьДокОснованиеКорректировки(Док, ТипДок) {
2      var Основание = null;
3      ДляВсех(Оснований(Док, ТипДок, ["Корректировка"]), function
(ДокОсн) {
4          Основание = ДокОсн;
5      });
6      var Основание = Основание || Док // нельзя повторно объявлять
переменные
7      return Основание;
8  }
```

- Код без ошибки

```
1  function ПолучитьДокОснованиеКорректировки(Док, ТипДок) {
2      var Основание = null;
```

```

3      ДляВсех(Оснований(Док, ТипДок, ["Корректировка"]), function
      (ДокОсн) {
4          Основание = ДокОсн;
5      });
6      Основание = Основание || Док
7      return Основание;
8  }

```

Перекрытие глобального имени

Тип: Ошибка

Название: no-shadow-restricted-names

[Источник](#)

Ошибка:

Переменные обозначены через глобальные объекты.

Решение:

Не используйте глобальные объекты для обозначения переменных:

- Arguments
- Infinity
- NaN
- Undefined

Пример:

- Код с ошибкой

```
1 function NaN() {}
```

- Код без ошибки

```
1 Не использовать глобальные объекты
```

Лишняя запятая в объявлении массива

Тип: Предупреждение

Название: no-sparse-arrays

[Источник](#)

Ошибка:

В массиве проставлены лишние запятые.

Решение:

Проверьте код и удалите лишние запятые, либо добавьте Нет.

Пример:

- Код с ошибкой

```

1 var АналитикиДебета = [ВидДоходаТС, , СтатьяТС]; // элементы массива
  нельзя пропускать
2 Проводка(ДатКнц, Счета.ПродажиРасходы, АналитикиДебета,
  Счета.ИздержкиОбращения, [СтатьяТС], - ВРасчете['ТоргСбор'], 0);

```

- Код без ошибки

```

1 var АналитикиДебета = [ВидДоходаТС, Нет, СтатьяТС]; // следует писать
  null явно
2 Проводка(ДатКнц, Счета.ПродажиРасходы, АналитикиДебета,
  Счета.ИздержкиОбращения, [СтатьяТС], - ВРасчете['ТоргСбор'], 0);

```

Перенос объекта на другую строку

Тип: Предупреждение

Название: no-unexpected-multiline

[Источник](#)

Ошибка:

Указана новая строка.

Решение:

Проверьте код и удалите лишний отступ.

Пример:

- Код с ошибкой

```
1 var пДата = ПлатежныеДокументы.Дата || Дата,
2   СчетБанка = Кошелек. // лишний перенос перед обращением к
   свойству
3   СчетУчета || Счета.РасчетныеСчета,
4   пСчетСоцСтрах = Счета.ОССВзносы;
```

- Код без ошибки

```
1 var пДата = ПлатежныеДокументы.Дата || Дата,
2   СчетБанка = Кошелек.СчетУчета || Счета.РасчетныеСчета,
3
4   пСчетСоцСтрах = Счета.ОССВзносы;
```

Неисполняемый код

Тип: Предупреждение

Название: no-unreachable

[Источник](#)

Ошибка:

После операторов перехода return, throw, break, continue указано выражение.

Решение:

Удалите неисполняемый элемент, либо поставьте выражение перед оператором перехода.

Пример:

- Код с ошибкой

```
1 function ПрошлыйБухОтчет(Годовой) {
2   var ПредОтчет = ОтчетПрошлогоПериода();
3   if (ПредОтчет.hasOwnProperty('ОтчетИзмКап')) {
4     delete ПредОтчет.ОтчетИзмКап;
5   }
6   return ПредОтчет; // рано объявлено завершение функции
7   if (ПредОтчет.hasOwnProperty('ДвижениеДен')) {
8     delete ПредОтчет.ДвижениеДен;
9   }
10 }
```

- Код без ошибки

```
1 function ПрошлыйБухОтчет(Годовой) {
2   var ПредОтчет = ОтчетПрошлогоПериода();
3   if (ПредОтчет.hasOwnProperty('ОтчетИзмКап')) {
4     delete ПредОтчет.ОтчетИзмКап;
5   }
6   if (ПредОтчет.hasOwnProperty('ДвижениеДен')) {
7     delete ПредОтчет.ДвижениеДен;
8   }
9 }
```

```
9 | return ПредОтчет;  
10 }
```

Параметр используется до объявления

Тип: Предупреждение

Название: no-use-before-define

[Источник](#)

Ошибка:

Использование переменной до ее объявления.

Решение:

Объявите переменную перед ее вызовом.

Пример:

- Код с ошибкой

```
1 var пДата = Дата,  
2     ВидПлатежа = НайтиАналитику('ВидыНП', 'НПОБ');  
3 мИсчисленНалог = ДанныеПоНалогуВРасчетеУСН(); // нельзя использовать  
   переменную до объявления  
4 var пКолПер = мИсчисленНалог.length,  
5     пСумПред = 0,  
6     пКвартал = 0,  
7     мИсчисленНалог; // переменная объявлена после использования  
8
```

- Код без ошибки

```
1 var пДата = Дата,  
2     ВидПлатежа = НайтиАналитику('ВидыНП', 'НПОБ');  
3 var мИсчисленНалог = ДанныеПоНалогуВРасчетеУСН();  
4 var пКолПер = мИсчисленНалог.length,  
5     пСумПред = 0,  
6     пКвартал = 0;
```

typeof сравнивается с неправильной строкой

Тип: Ошибка

Название: "valid-typeof": 2

[Источник](#)

Ошибка:

Опечатка в значении, с которым сравнивается результат оператора typeof.

Решение:

Исправьте опечатку.

Пример:

- Код с ошибкой

```
1 var AccountNum = (typeof (Счет) == 'strings') ? Счет :  
   ПолучитьНомерСчетаПоИД(Счет); // Неправильно название типа string  
2 ДляВсех(Объект, function (СальдоОС) {  
3     if (typeof resObj[СальдоОС.Лицо2.Ид0] == 'undefind') { //  
       Неправильно название типа undefined  
4         resObj[СальдоОС.Лицо2.Ид0] = {};  
5     }  
6     resObj[СальдоОС.Лицо2.Ид0][СальдоОС.Лицо1.Ид0] =  
       СальдоОС[ТипСальдо];  
7 });
```

- Код без ошибки

```
1 var AccountNum = (typeof (Счет) == 'string') ? Счет :  
  ПолучитьНомерСчетаПоИД(Счет);  
2 ДляВсех(Объект, function (СальдоОС) {  
3     if (typeof resObj[СальдоОС.Лицо2.Ид0] == 'undefined') {  
4  
5         resObj[СальдоОС.Лицо2.Ид0] = {};  
6     }  
7     resObj[СальдоОС.Лицо2.Ид0][СальдоОС.Лицо1.Ид0] =  
  СальдоОС[ТипСальдо];  
8 });
```

Использовать только одинарные кавычки во всем коде

Тип: Предупреждение

Название: quotes

[Источник](#)

Ошибка:

В коде использованы двойные кавычки.

Решение:

Заменить двойные кавычки на одинарные.

Пример:

- Код с ошибкой

```
1 var single = "single";
```

- Код без ошибки

```
1 var single = 'single'
```

Переменные необходимо использовать в пределах блока, в котором они были определены

Тип: Ошибка

Название: block-scoped-var

[Источник](#)

Ошибка:

Переменная используется вне блока, в котором она определена.

Решение:

Вынести определение переменной выше, либо использовать переменную внутри блока.

Пример:

- Код с ошибкой

```
1 function doIf() {  
2     if (true) {  
3         var build = true;  
4     }  
5     console.log(build);  
6 }
```

- Код без ошибки


```
1 function doIf() {
2     var build;
3     if (true) {
4         build = true;
5     }
6     console.log(build);
7 }
```

Неиспользуемые переменные запрещены

Тип: Ошибка

Название: no-unused-vars

[Источник](#)

Ошибка:

Переменная была объявлена, но нигде не использована.

Решение:

Удалите неиспользуемые переменные.

Пример:

- Код с ошибкой

```
1 var x;
```

- Код без ошибки

```
1 var x = 10;
2 alert(x);
```

Запрещены строки длиннее 120 символов

Тип: Предупреждение

Название: max-len

[Источник](#)

Ошибка:

Строка длиннее 120 символов.

Решение:

Сократите длину строки.

Запрещено более двух пустых строк подряд

Тип: Предупреждение

Название: no-multiple-empty-lines

[Источник](#)

Ошибка:

Между строками кода три пустые строки.

Решение:

Сократить количество пустых строк до двух.

Пример:

- Код с ошибкой

```
1 var foo = 5;
2
3
4
```

```
5 | var bar = 3;
```

- Код без ошибки

```
1 | var foo = 5;  
2 |  
3 |  
4 | var bar = 3;
```

Запрещено размещение комментария в строке с кодом

Тип: Ошибка

Название: line-comment-position

[Источник](#)

Ошибка:

Комментарий находится в строке с кодом.

Решение:

Разместить комментарий над строкой с кодом.

Пример:

- Код с ошибкой

```
1 | + 1; // invalid comment
```

- Код без ошибки

```
1 | // valid comment  
2 | 1 + 1;
```

Пропущены фигурные скобки после оператора

Тип: Ошибка

Название: curly

[Источник](#)

Ошибка:

Нет фигурных скобок после операторов if, else, for, while или do.

Решение:

Когда в блоке есть несколько из перечисленных операторов, после каждого необходимо использовать фигурные скобки.

Пример:

- Код с ошибкой

```
1 | if (foo) {  
2 |     foo++;  
3 | }  
4 |  
5 | if (foo) bar();  
6 | else {  
7 |     foo++;  
8 | }  
9 |  
10 | while (true) {  
11 |     doSomething();  
12 | }  
13 |
```

```
14 | for (var i=0; i < items.length; i++) {  
15 |     doSomething();  
16 | }
```

- Код без ошибки

```
1 | if (foo) foo++;  
2 |  
3 | else foo();  
4 |  
5 | while (true) {  
6 |     doSomething();  
7 |     doSomethingElse();  
8 | }
```